Revature
2026 / October

P2: Bank of CLI Full Stack

Group 3

Contents

1. Full-Stack Integration
2. Architecture
3. Demos
4. Questions?

Full-Stack Integration

Communication

HTTP communication between the frontend and backend

1

Accounts

Users register and login through /api/auth. Backend validates the request and returns the account details along with a JWT. Frontend sends that token with every request to prove who the user is.

2

Transactions

Transactions are executed depending on the type the user chooses. Calls the Transaction Service and returns the completed transaction.

3

Transaction History

The backend returns an updated list of past transactions to display in a table on the dashboard.

4

Balance

Balance is returned to the frontend along with transaction responses instead of being calculated locally, to remain in sync with the database.

JWT Security

1	User Authentication	Users log in using their registered email and passwords.
2	Password Protection	Passwords are securely hashed rather than stored in plaintext.
3	JWT Generation	After successful login, the backend generates a signed JSON Web Token.
4	Protected API Endpoints	The frontend includes the JWT in Authorization header when accessing secured endpoints.
5	Stateless Authentication	Spring Security validates the token, allowing users to access protected banking features without maintaining server-side login sessions.

Access Security

Secure Registration

Validates user information before account creation.

Prevents duplicate accounts by checking registered email addresses.

Uses BCrypt to hash passwords before storing them in the database

Secure Login

Authenticates users using their registered email and password.

Uses Spring Security's **PasswordEncoder** to compare passwords against stored hashes.

Rejects invalid Credentials to prevent unauthorized access.

Testing

We wrote a simple test for the authorization services using Mockito.



Architecture

Tech Stack

Database: AWS, ECS, PostgreSQL

Hosts the database online via Cloudfront instead of locally

Backend: Spring Boot (Java)

Automatically configures dependencies with Maven

Security: JWT (JSON Web Tokens)

Stateless, scalable, decentralized verification of user identity

Communication: REST / JSON

Standard, stateless HTTP methods for CRUD, and allows client-server decoupling

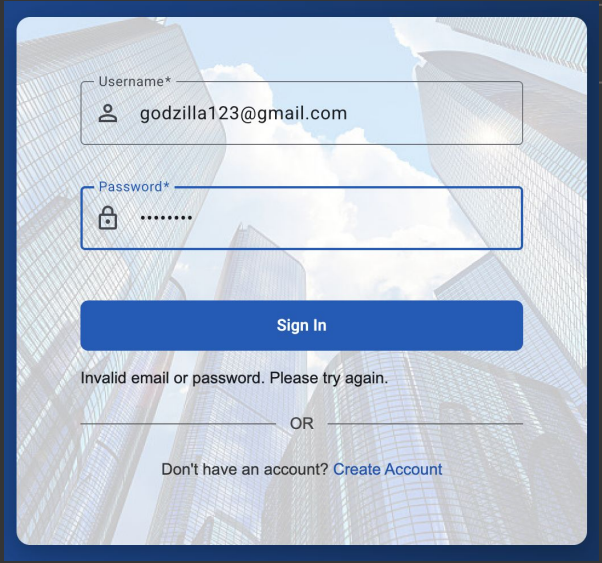
Frontend: React and Angular (TypeScript)

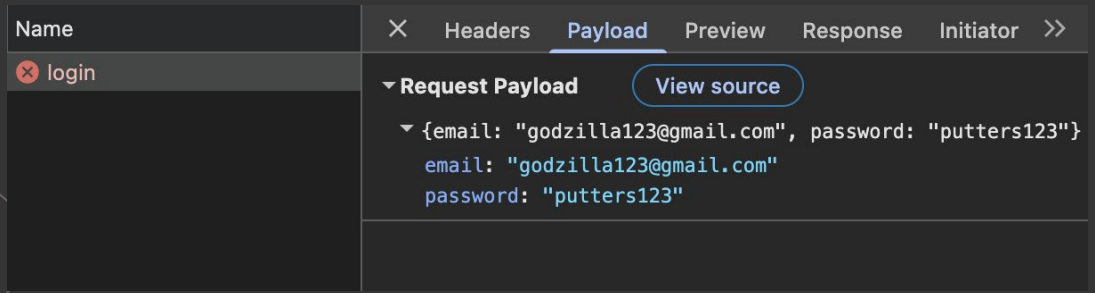
The two frontends from P1

Both connect to the same backend

Angular Integration

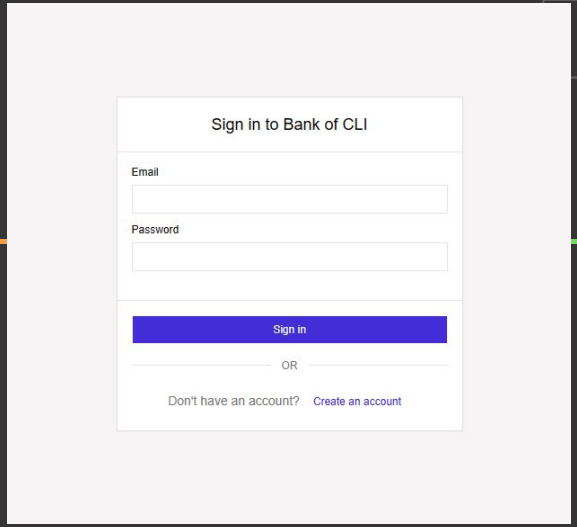
- Refactored login, registration, and transfers to communicate with the Spring Boot REST API using Angular's HttpClient
- Centralized HTTP requests in a shared injectable service, using Observables to handle backend responses





React Integration

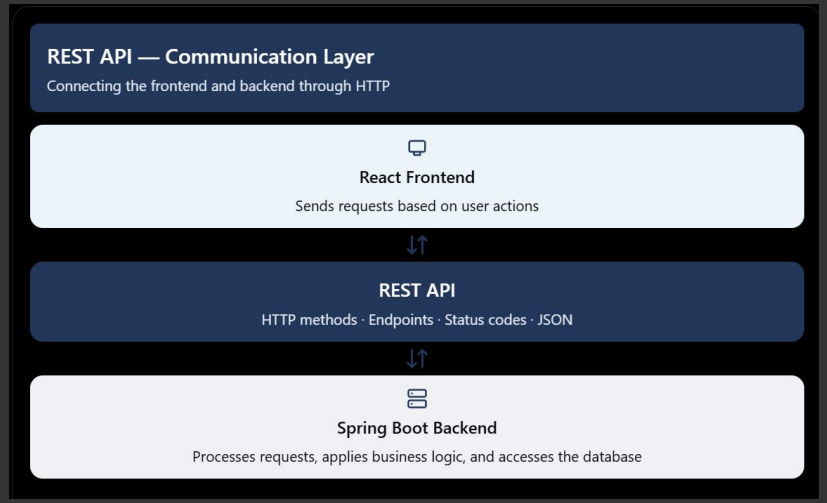
- Mock database calls in our React project were refactored to route through our REST endpoints using the fetch API
- One file stores and exposes all API-related functionality, making the logic more maintainable
- These methods are imported as needed into components




```
export const register = (body: RegisterRequest) => api<Account>("/accounts", { method: "POST", body });

export async function login(email: string, password: string): Promise<void> {
  const { token } = await api<{ token: string }>("/auth/login", {
    method: "POST",
    body: { email, password },
  });
  setToken(token);
}
```

REST API



Concepts

- HTTP Methods: GET retrieves data, POST creates data, PUT/PATCH updates data, and DELETE removes data.
- Endpoints: URLs identify resources, such as /api/accounts or /api/account/[transaction_type].
- JSON: A common format for exchanging structured data between the frontend and backend.
- HTTP Status Codes: Communicate results, such as 200 (OK), 201 (Created), 400 (Bad Request), and 404 (Not Found).

Spring Boot

Transaction Service, Exceptions, Domain And Repository

Transaction Service

Handles valid deposit, withdrawal, and transfer transactions. Uses @Transactional to enforce ACID on transactions.

Repository

Contains Interfaces used to persist accounts and transactions to the database using Spring Data JPA (Hibernate).

Global Exception Handler

Catches exceptions from any controller, including validation errors and unexpected runtime errors. Returns the correct HTTP status and error body.

Domain

Contains entities and the relationships between them, mapped to the database tables using JPA annotations such as @Entity and @Table. Example entities include transaction and types of transactions.

Row Locking

Prevents lost updates when two requests hit the same account at once. Each transaction locks the account row, and transfers lock the lower account ID first to avoid deadlocks.

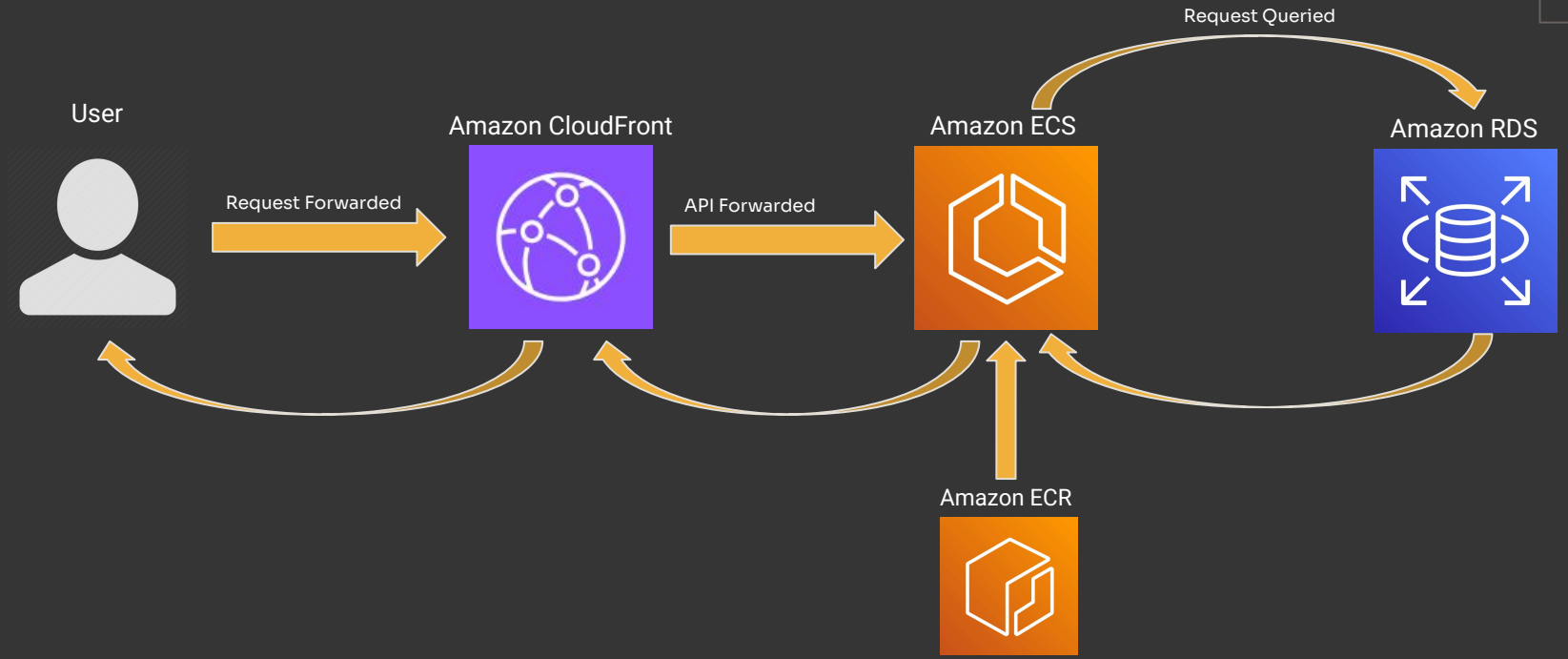
Custom Exceptions

Represent failed or invalid transactions. Each one carries its HTTP status and contract message and they are thrown from the service layer.

15 JUnit 5 and Mockito unit tests cover deposits, withdrawals and transfers. MockMvc tests confirm that each error returns the correct status and contract message. CI runs every test on each pull request.

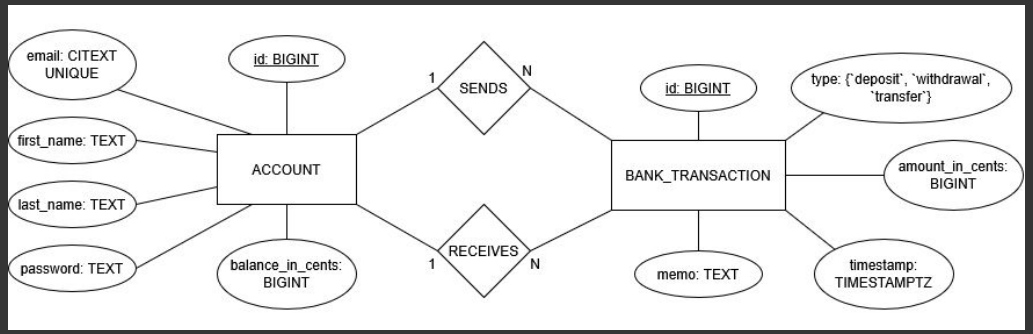
AWS

Hosting Database And Deploying Application



Database Schema

The structure of our PostgreSQL database



Account

- ID (auto-incremented)
- Email (unique and case-insensitive)
- First and last name
- Hashed password
- Balance in cents (default 0, never negative)

Bank Transaction

- ID (auto-incremented)
- Type ('deposit', 'withdrawal', or 'transfer')
- Money amount in cents (always positive)
- Timestamp
- Memo (for use in the future)
- Sender and recipient accounts

```
CONSTRAINT account_id_foreign_keys_match_type CHECK (
    CASE
        -- Deposit
        WHEN type='deposit'
            THEN sender_account_id IS NULL
            AND recipient_account_id IS NOT NULL

        -- Withdrawal
        WHEN type='withdrawal'
            THEN sender_account_id IS NOT NULL
            AND recipient_account_id IS NULL

        -- Transfer
        WHEN type='transfer'
            THEN sender_account_id IS NOT NULL
            AND recipient_account_id IS NOT NULL

        ELSE FALSE
    END
),
```

Bank Transaction Constraints

- Last line of defense for data integrity
- Relationships NULLed depending on `type`
- Self-transfers are forbidden
- Bank transactions handle $10,000 maximum

Demos

Questions?